1 Scope

The question is whether a learning oracle can answer training and evaluation requests in unit time without changing any adversary's view. A second question is where such an oracle becomes useless and where it solves the adversary's problem outright. We state the problem for any procedure that trains or queries models, then give a definition, three proven properties and an approximation by a model bank.

2 Result

An oracle that samples exactly from the learning function, costs one unit per call, takes a record-indexed seed and obeys an access policy is well defined and lies between useless and trivial. Replacing every training and evaluation step by an oracle call leaves every adversary's view unchanged in law, running time excluded (proven, Proposition 1). Naming an honest dataset with its seed decides membership in one call, and naming only one's own data gains nothing without a call budget (proven, Propositions 2 and 3). The model's content therefore lies in the call budget and the access policy, and one policy, neighbouring datasets with secret seeds, is the adversary of differential privacy. For the experimenter, who may name everything, the oracle turns a security game into a sampling procedure with two forms of the leave-one-out model. A bank of models trained ahead of time answers every call it covers exactly, and errs in coverage, sampling and hardware nondeterminism.

Does not show: This is a definition with proof sketches, and no experiment has run. We have not checked its novelty against three lines of work, all cited from memory and not placed. Oracle-efficient learning assumes an empirical risk minimization oracle (Agarwal et al., ICML 2014). Membership attacks train shadow models (Carlini et al., IEEE S&P 2022), and datamodels fit models on many subsets (Ilyas et al., ICML 2022). A result in the oracle model transfers to a real procedure only as far as that procedure exposes its node values and no more. The random-oracle model carries the same caveat (Canetti, Goldreich and Halevi, STOC 1998, from memory).

3 How it was produced

Sources	Halil's statement of the idea in the session of 2026-10-16, and the adversary interface of `lib/reports/2026-10-16-halil-adversary-interface.pdf`
Artifacts	this report alone
Reproduce	no code, the source compiles on its own

4 What this changes

The protocol graph of the interface report takes its canonical name, the dataflow graph, and a semantics in which each learning node is one oracle call. A question about an adversary becomes a question about its call budget under an access policy, and an experiment becomes a replay of the dataflow graph against a model bank. Filling the bank takes as many sequential waves of training as the graph has learning nodes on its longest path, and each wave trains in parallel. The next steps are to fix the access policy of each setting we study and to measure on one small procedure how a membership estimate converges with the size of the bank.

5 Method

The problem

Many systems train and use models as steps of a larger procedure. A company trains a classifier on its users' records and answers prediction queries. A group of hospitals trains one model across sites. A provider deletes a user's records and must show that its model forgot them. A data market pays each owner by how much its data improved a model. Each procedure is a sequence of steps, and

each step trains a model, queries a model, or computes something ordinary, such as an average, a ciphertext, a noise draw or a release. Training is a learning function, which maps a starting model, a dataset and a hypothesis class to a model. Querying is an evaluation map, which maps a model and an input to an output. We draw the procedure as a directed acyclic graph whose nodes are steps and whose edges carry datasets and models, and we call it the procedure's *dataflow graph*, the name the object has in systems work.

An adversary sees some node values, each through a channel: the plain value, a sum, a ciphertext, a noisy value, or nothing. It sets some inputs and pursues one of three goals. It distinguishes two hypotheses about the datasets, inverts a target, or steers the trained model. A proof or a measurement of its success needs the law of what it sees, and training blocks both. No closed form gives the law of a trained model. A proof therefore treats training as an arbitrary function and loses everything specific to learning, or it assumes a structure that real training lacks, such as a convex loss. Every sample of that law costs a full training run, and a membership advantage needs many runs per target record. Experiments shrink as a result. 215 of the 349 papers of our sweep evaluate on MNIST or CIFAR-10.

The idea is the one cryptography applied to hash functions in the random-oracle model (Bellare and Rogaway, CCS 1993, from memory). An oracle replaces the learning function. It answers a training request in one unit of time, with a model distributed exactly as real training would produce it, and it answers evaluation requests the same way. A caller that may name any sub-dataset and fix the randomness obtains, for instance, the model trained on everything except record j under the randomness of the original run, written $m_{\bar{x}_j}$. A procedure can then be specified and analysed step by step without the inside of training, and simulated by looking answers up. The oracle must not change what the adversary sees when it replaces real training.

Defined loosely, the oracle is trivial or useless. The oracle is trivial when the adversary may request the model trained on an honest dataset without record j under the honest randomness. Comparing that model with the released one decides membership. The oracle is useless when it computes only what the adversary could compute itself. The best attack of an adversary without resource limits depends on the law of its view alone, and such an oracle leaves that law unchanged. The problem is to fix every property that decides between these two failures: what the oracle computes, whose randomness it uses, what a call costs, who may name which datasets, which models a caller may open or only query, and what the oracle refuses. Such an oracle cannot exist, because training takes time. We approximate it by training the needed models ahead of time and asynchronously, storing them in a bank keyed by dataset and seed, and replaying the procedure against the bank.

The setting

Let $\mathcal{Z}$ be a data space. A *pool* is a finite sequence $P = (z_1, \dots, z_n)$ of records, and a reference $I \subseteq [n]$ names the dataset $P_I = \{z_i : i \in I\}$. A dataset may also be given by value, as a finite multiset over $\mathcal{Z}$. Each record of any dataset carries a key, its index in the pool or a fresh key assigned when the dataset is submitted. Let $\mathcal{M}$ be a model space and $\mathbb{H}$ a finite family of named hypothesis classes $\mathcal{H} \subseteq \mathcal{M}$. A learning function is a map $\mathsf{L}(m, D, \mathcal{H}; \rho) \in \mathcal{H}$ of a starting model, a dataset, a class and a seed $\rho \in \mathcal{R}$ with law μ. The randomized learning function of the interface report is the law of $\mathsf{L}(m, D, \mathcal{H}; \rho)$ under $\rho \sim \mu$. An evaluation map is $\mathsf{ev}(m, u; \sigma)$ with a seed $\sigma \sim \nu$.

The seed is indexed by record. Write $\rho = (\rho_0, (\rho_k)_{k \in K})$ over the set K of keys, with $\rho_0 \sim \mu_0$ and the ρ_k independent with law μ_1. The learning function is *local* when it uses ρ_k only in decisions about record k and ρ_0 for everything else, such as the initialization and noise not tied to a record. Poisson subsampling, where record k enters step t when coordinate t of ρ_k falls below the rate, is local. A shuffle that sorts records by a key drawn from ρ_k is local. A shuffle by one permutation of positions is not, since removing one record moves every later one.

A procedure is a dataflow graph $G = (V, E)$. Each node has an owner and one of four operations. An input node holds a dataset or a public object. A learning node computes $\mathsf{L}(\cdot, \cdot, \mathcal{H}_v; \rho_v)$ and an evaluation node $\mathsf{ev}(\cdot, u_v; \sigma_v)$. A general node computes a randomized function, such as an

aggregation, an encryption, a noise draw or a release. An execution evaluates the nodes in topological order, with independent seeds per node unless the procedure shares them. An adversary is a coalition with a view, choices and a goal, as in the interface report.

The oracle

Definition 1 (learning oracle). Fix (L, μ), (ev, ν), a pool P and the family $\mathbb{H}$. A learning oracle $\mathcal{O}$ keeps a table T from handles to models and answers three calls.

- $\mathsf{Train}(a, D, \mathcal{H}, \rho)$, with a a model or a handle, D a reference or a dataset by value, $\mathcal{H} \in \mathbb{H}$, and ρ a seed or $\perp$. On $\perp$, $\mathcal{O}$ draws $\rho \sim \mu$ and keeps it. $\mathcal{O}$ stores $\mathsf{L}(a, D, \mathcal{H}; \rho)$ under a fresh handle h and returns h.
- $\mathsf{Eval}(h, u, \sigma)$ returns $\mathsf{ev}(T[h], u; \sigma)$, drawing $\sigma \sim \nu$ on $\perp$.
- $\mathsf{Open}(h)$ returns $T[h]$.

It satisfies six properties.

- Fidelity. Every answer equals the learning function or the evaluation map at its seed, and answers with drawn seeds are independent given their inputs.
- Consistency. Equal calls with equal explicit seeds return equal models.
- Locality. The learning function is local in the record-indexed seed.
- Unit cost. A call costs one, whatever the dataset size, class or model.
- Access. Each caller c holds a policy Π_c listing the references it may name, the seeds it may pass (its own and those Π_c makes public), the handles it may open, and optionally a sampler of fresh records from a data distribution. A handle reaches a caller only as a value it sees, and a caller may evaluate every handle it holds. $\mathcal{O}$ refuses calls outside the policy.
- Closure. $\mathcal{O}$ answers no other call. It reveals no seed it drew, no state inside training, no preimage of a model, and no running time.

A caller that holds a handle it may not open has black-box access, and Open gives white-box access. A caller's Eval calls are counted apart from its Train calls, as the interface report counts label queries against the allowance Q.

Definition 2 (learning-oracle model). The procedure $G^{\mathcal{O}}$ makes each learning node a Train call and each evaluation node an Eval call by the node's owner, with seeds drawn by $\mathcal{O}$ unless the procedure fixes them. An adversary is a coalition with a policy Π and a budget (q_T, q_E) of its own Train and Eval calls. The experimenter holds the full policy, every reference, every seed and every handle.

Three properties

Proposition 1 (perfect substitution). For every procedure G, inputs and adversary, the view in G with L and ev computed directly has the same law as the view in $G^{\mathcal{O}}$, when the view excludes running time. Proven, by induction over a topological order of G. By fidelity each node value has the same law given its parents in both executions, and seeds of different nodes are independent in both. The adversary's own calls have the law of the computation it would run itself, by fidelity, and closure keeps every other value out of the view. The exclusion of time is necessary, because unit cost makes a call's time independent of the dataset size and real training does not.

Proposition 2 (reference access). Let the honest learning node train on $P_{I\cup\{j\}}$ under H_1 and on P_I under H_0, with a seed ρ drawn by $\mathcal{O}$, and let the adversary see the model.

(a) If Π lets it name I and pass ρ, and $\mathsf{L}(m, P_I, \mathcal{H}; \rho) \neq \mathsf{L}(m, P_{I\cup\{j\}}, \mathcal{H}; \rho)$, it wins with advantage 1 and one call.

(b) If Π lets it name I and $I \cup \{j\}$ with drawn seeds only, its advantage is at most $\mathrm{TV}(\mathsf{View} \mid H_0, \mathsf{View} \mid H_1)$ for every budget. When the view takes finitely many values, a likelihood-ratio test built from q_T samples per hypothesis reaches this bound as $q_T \to \infty$.

Proven. Part (a) compares two models. In part (b) the bound is the data-processing inequality, and the test is optimal by Neyman and Pearson once its empirical frequencies converge, which they do almost surely. Policy (b) is the adversary of differential privacy, which knows both neighbouring datasets and not the seed. The oracle model contains that threat model as one policy, and a differentially private learning function bounds the advantage of every adversary under it.

Proposition 3 (own data only). If Π names no reference and passes only seeds the adversary drew, an adversary without a budget gains nothing from $\mathcal{O}$. Proven, since every answer is a value of L or ev on inputs it holds. A result in the model is therefore an advantage as a function of (q_T, q_E) under a stated policy. Shadow-model membership attacks are adversaries of this kind, with a sampler of records from a data distribution and q_T shadow models.

Two leave-one-out models

Let $m = \mathsf{L}(m_0, P_I, \mathcal{H}; \rho)$ with $j \in I$. The coupled leave-one-out model is $m_{\bar{x}_j} = \mathsf{L}(m_0, P_{I \setminus \{j\}}, \mathcal{H}; \rho)$, under the same seed. The independent one is $m'_{\bar{x}_j} = \mathsf{L}(m_0, P_{I \setminus \{j\}}, \mathcal{H}; \rho')$, under a fresh seed ρ'. The two have the same law and differ in their joint law with m. Under locality the coupled pair differs only through the participation of record j. It therefore measures what record j did in this run, the object of influence and memorization measures. The independent pair compares two laws, which a membership game depends on and differential privacy bounds. Without locality the coupled form loses its meaning, because removing a record changes every minibatch.

The experimenter uses the oracle in three ways. It estimates an adversary's advantage by drawing the hypothesis, running $G^{\mathcal{O}}$ and the adversary, and repeating, with a binomial confidence interval. It tests a claimed equality of laws, such as exact unlearning, which claims that the unlearned model has the law of $m'_{\bar{x}_j}$. It computes models on many subsets, which data valuation needs.

Approximation by a model bank

Definition 3 (model bank). A design Δ is a finite set of Train calls with explicit seeds. The bank B for Δ stores the model of each call, computed by running L. The empirical oracle $\widehat{\mathcal{O}}_B$ answers a call in Δ by lookup, and Eval and Open on stored models. It refuses a call outside Δ in a fixed-design experiment, and queues it as a training job in an asynchronous one.

Every answer of $\widehat{\mathcal{O}}_B$ is a sample of L at its seed. Fidelity therefore holds on Δ by construction, and unit cost holds once the bank is paid for. The bank errs in three ways. Consistency needs deterministic training, and GPU kernels are nondeterministic by default. Coupled pairs degrade unless the training enforces deterministic kernels. An adversary whose calls depend on earlier answers may leave Δ. A fixed bank therefore serves non-adaptive adversaries, and the asynchronous mode charges adaptivity in waiting time. An estimate from N executions carries a sampling error of order $N^{-1/2}$, and a true-positive rate at a low false-positive rate needs far more executions.

A design that includes each record independently with probability one half in each of k runs gives every record about $k/2$ runs with it and $k/2$ without. One bank thus serves all n records (the design of Carlini et al., from memory). Exact leave-one-out costs $n + 1$ runs per seed. A learning node whose starting model is another learning node's output waits for it. With enough workers one execution fills in as many waves as its longest path has learning nodes, and each wave trains in parallel. Federated averaging over T rounds and N clients takes T waves of N jobs, and a one-shot procedure takes one wave.

6 Next

The open choices, each a decision for us.

- The access policy of each setting we study, from records by value for a client to neighbouring

references for the adversary of differential privacy.

- The cost unit, one per call or proportional to the dataset size, which prices shadow training on large datasets.
- The granularity of a learning node, one node per optimizer step, which puts the iterates in the graph, or one node per training run, which hides them.
- Adaptive adversaries against a bank, and how long they may wait.
- Deterministic training on the cluster, which consistency requires.
- A public pretrained model as an input node with its own bank entry.